

Context Harvester — PRD

Oct 8, 2026 · @roo

Summary

Context Harvester takes a repository and an issue, and returns the ranked minimal set of files a coding agent needs to read in order to fix it.

The output is a ranked file list with a per-file reason and a token budget. The claim it makes is measurable: file-level recall@10 against the files the merged fix actually touched, compared against a BM25 baseline and a naive embedding baseline.

Ground truth is not hand-labeled. It comes from SWE-bench Verified (500 human-validated instances with gold patches) and from a harvester that mines Fixes #N issue-to-PR links out of any public repository.

The evaluation corpus is assembled in Clay on a 14-day Growth trial and committed to the repo as a CSV. Everything after that runs on GitHub Actions against public repos, stores results as Parquet, and serves a UI from GitHub Pages. Total infrastructure cost is zero, and nothing in the demo path expires.

Problem

Coding agents fail on large repositories because of what they read, not because of how they write.

The constraint is concrete. An agent has a finite context budget. A monorepo has tens of thousands of files. Something has to decide which files enter the window, and that decision determines whether the agent produces a correct patch or a confident wrong one. At scale the same decision is also the main cost driver, since tokens read are tokens paid for.

Why this is Clay's problem specifically

Clay's developer experience team lists "expose rich codebase context to coding agents" as a responsibility, alongside supporting parallel agents running automated refactors, migrations, and bug fixes. Parallel agents multiply the context problem rather than dividing it: every agent in flight is independently choosing what to read, against the same monorepo.

What existing approaches miss

- **Whole-repo stuffing** does not fit, and degrades as irrelevant files crowd the window.
- **Lexical search (BM25)** is a genuinely strong baseline on code and is often underrated. It fails when the issue describes a symptom in user language and the fix lives in code that shares no vocabulary with it.
- **Naive embedding retrieval** handles the vocabulary gap but ignores structure. A file can be semantically unrelated to the issue text and still be required reading, because the file that must change imports from it.
- **Agent-driven exploration** (let the agent grep around) works but burns turns and tokens, and the cost is invisible until the bill arrives.

The gap is a retrieval system that combines all three signals and reports a number proving it beat the baselines.

Goals and non-goals

Goals

1. Given a repo URL and an issue number, emit a ranked file list with per-file reasons and a running token budget, in under 60 seconds on a cold repo.
2. Beat BM25 on file-level recall@10, measured on held-out instances, with the margin reported rather than asserted.
3. Run the same metric on two independent ground-truth sources: SWE-bench Verified, and self-harvested issue-to-PR links from repos SWE-bench never covered.
4. Publish the full ablation, including the configurations that lost.
5. Cost nothing to run and stay live indefinitely, so the demo works months after it was built.

Non-goals

- **Writing the patch.** This selects context. It does not attempt the fix. Patch generation is a solved-enough demo and would dilute the measurable claim.
- **Beating SWE-bench resolve rates.** The metric here is retrieval quality, not end-to-end task success.
- **Supporting every language.** Python first, because SWE-bench Verified is Python and the import graph is tractable. TypeScript second if time allows.

- **A production service.** No multi-tenancy, no auth, no SLA. It is a CLI plus a static results page.
- **A live Clay dependency.** The Growth trial is used to build the evaluation corpus, but nothing in the demo path calls Clay at runtime. See the Clay integration section.

Users and the demo

Primary user

An engineer on a platform or DX team who runs coding agents against a large repo and wants to control what those agents read. Secondary user: the agent itself, calling the ranker as a tool before it starts work.

The 60-second demo

This is the shape of the interview walkthrough. The interviewer picks the input, which is the point.

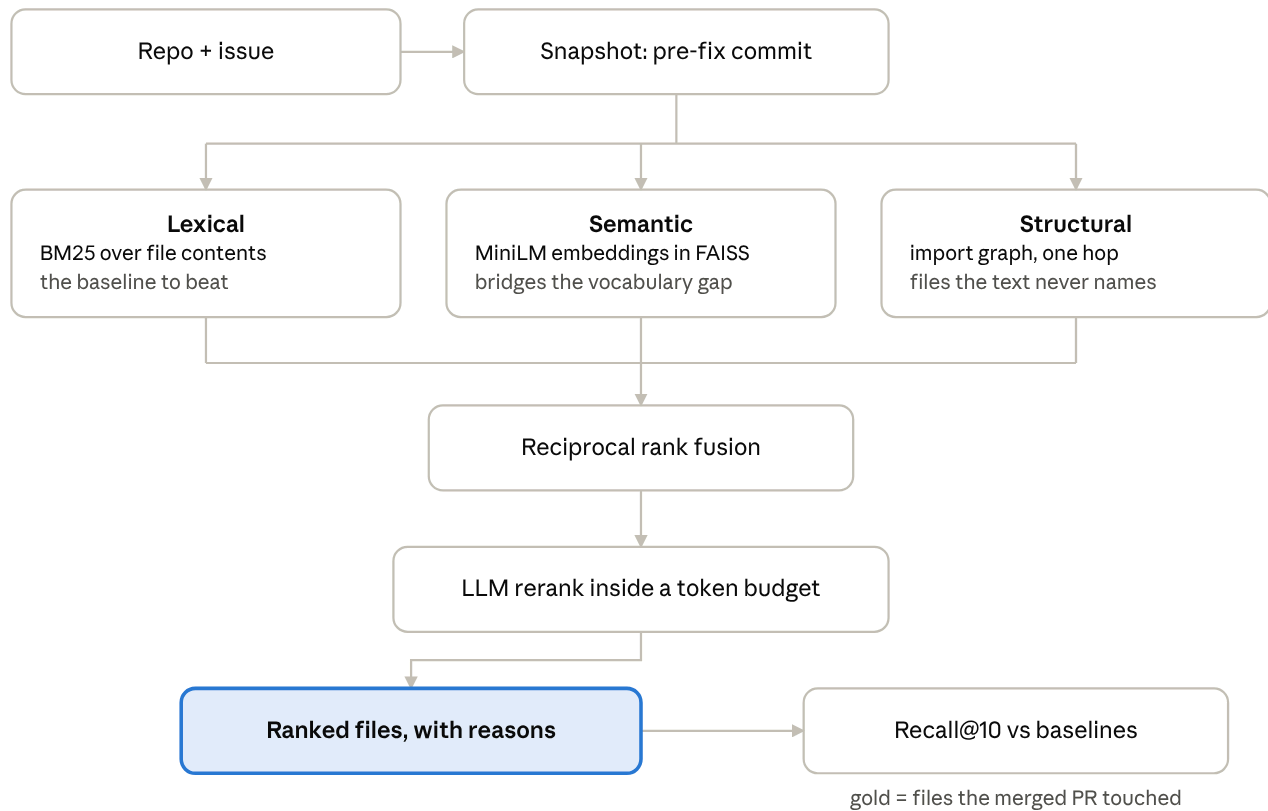
1. **They name a public repo and an open issue number.** Any repo, not a prepared one.
2. **The CLI runs.** It clones or fetches, builds or loads an index, and prints a ranked file list with a one-line reason per file and a cumulative token count.
3. **Then the replay.** The same CLI runs against a *closed* issue from that repo, where the merged fix is known, and shows which of the truly-changed files it surfaced at k=10, next to what BM25 alone would have surfaced.
4. **The results page.** A static page with the full benchmark: recall@k curves across all evaluated repos, the ablation table, and the per-configuration numbers.

Step 3 is what distinguishes this from a retrieval demo. The system grades itself live, on a repo it was not tuned for, against ground truth it did not author.

How it works

Three signals merge into one ranked list, scored against the real fix

Context Harvester pipeline



Context Harvester pipeline · 3 retrieval signals, 1 scored output

The three retrievers run independently and are fused, so each one's contribution can be measured by removing it.

Stage detail

1. **Snapshot.** Check out the commit immediately before the fix with `git worktree`. Everything downstream indexes that snapshot, never current `main`.
2. **Lexical.** BM25 over file contents and paths. Fast, no model, and the bar everything else has to clear.
3. **Semantic.** One embedding per file (chunked and pooled for large files), searched with FAISS in-process. This is what handles an issue written in user language against code that shares none of its words.

4. **Structural.** Parse imports into a graph. Expand one hop from the top lexical and semantic hits, weighted below direct matches. This is the step that surfaces the interface a changed file implements, which neither text method will find.
5. **Fusion.** Reciprocal rank fusion across the three lists. Chosen over a learned combiner because it needs no training data and cannot overfit the small evaluation set.
6. **Rerank.** A single LLM pass over the top candidates, which drops files and writes the one-line reason per file. Stops at the token budget rather than a fixed file count.
7. **Score.** In evaluation mode, compare the ranked list against the files the merged PR touched.

The LLM appears once, at the end, on a short list. It is not doing the retrieval.

Evaluation design

This section is the project. The retrieval code is ordinary; the thing that makes it credible is that the number is defensible.

Ground truth

Two independent sources, neither hand-labeled.

Source	What it gives	Size	Why it matters
SWE-bench Verified	Issue text plus the gold patch, human-validated	500 instances, 12 Python repos	Recognized benchmark, already leakage-controlled, removes the self-labeling objection
Self-harvested Fixes #N links	Issue text plus the merged PR's changed-file set	As many repos as time allows	Generalizes past those 12 repos, including to repos the interviewer names

The gold label in both cases is the set of files the merged fix touched. That label is causally correct: those files genuinely were the fix.

Metrics

- **Primary:** file-level recall@10.
- **Reported alongside:** recall@5, recall@20, and recall at a fixed token budget (more useful than recall at a fixed file count, since files vary wildly in size).

- **Cost axis:** tokens required to reach a given recall. A method that hits 0.8 recall in 40k tokens beats one that hits 0.82 in 200k.

Baselines

Every number is reported against these, not in isolation.

1. **Random file selection**, as a floor.
2. **BM25 over file contents**. The real bar. Build this in week one.
3. **Naive embedding retrieval**, single-vector per file, no structure.
4. **BM25 plus embeddings**, reciprocal rank fusion, no import graph. This isolates what the graph expansion actually contributes.

Protocol and leakage guards

These are the details that make or break the number, and the ones an interviewer will probe.

- **Snapshot the repo as of the issue date.** Use `git worktree` to check out the commit immediately before the fix. Indexing current `main` means the fix is already in the index, and every number is meaningless.
- **Forward time split.** Train or tune on issues before a cutoff date, evaluate on issues after it. No random shuffling.
- **Strip the fix from the issue text.** Some issues contain a suggested patch or a traceback naming the exact file. Report numbers with and without those instances, since they inflate the result.
- **Filter incidental files from the gold set.** Lockfiles, changelogs, generated code, version bumps, and test fixtures are touched by the fix but are not what the agent needs to understand. Report both filtered and unfiltered recall, and state the filter rules explicitly.
- **Hold out repos, not just issues.** At least two repos appear only at evaluation time, so the result is not tuned per repo.

The caveat to state unprompted

Recall against changed files is a **lower bound** on the true target. An agent must read more than it edits: the file it changes may only make sense alongside the interface it implements or the caller it breaks. So a file ranked highly but absent from the gold set is not necessarily a false positive.

Saying this out loud, before anyone asks, is what separates a real evaluation from a self-graded one.

Architecture and stack

Every component below is free and chosen for a reason that can be defended in two sentences. Nothing is included to match a line in a job description. Clay is the one time-limited piece, and it sits in a build step whose output is committed, never in the demo path.

Component	Choice	Reason
Corpus selection	Clay, on the Growth trial, exported to CSV	Filtering entities by enriched attributes is what Clay is built for. Build once, export, commit: no runtime dependency on a trial that expires.
Compute	GitHub Actions on a public repo	Free and uncapped for public repos. The collector is a scheduled job, which is what Actions is for.
Language	Python	SWE-bench is Python, the AST and import-graph tooling is Python, and the eval harness is data work.
CLI	Python with <code>typer</code>	The demo surface. One command, no server to wake up.
Lexical index	BM25 via <code>rank_bm25</code> or Tantivy	The baseline must be in-process and fast. No service dependency.
Embeddings	<code>all-MiniLM-L6-v2</code> locally, or a free API tier as fallback	Runs on a laptop, costs nothing, and removes a network dependency from the demo path.
Vector store	FAISS, in-process	A few thousand files per repo. Anything hosted would be infrastructure for its own sake.

Component	Choice	Reason
Results storage	Parquet committed to the repo	Versioned with the code, readable by the page, nothing to keep alive.
UI	Static page on GitHub Pages	Loads instantly for whoever opens the link, months from now.
LLM calls (reranking only)	Gemini Flash-Lite batch, Mistral free tier, Groq, behind a provider-fallback layer	Free tiers shift. The fallback layer is both the practical answer and a reasonable thing to show.

Deliberately not used

Stating these, with reasons, is better than quietly omitting them.

- **No hosted vector database.** The corpus is a few thousand files. FAISS in-process is correct; Pinecone or OpenSearch here would be a cost with no benefit.
- **No Redis.** Nothing in this system has request volume worth caching.
- **No Fargate, Aurora, ElastiCache, or managed OpenSearch.** None has a free tier, and new AWS accounts now close after six months. A demo that dies mid-job-search is worse than no demo.
- **No Terraform.** There is no cloud infrastructure to provision. Writing IaC for infrastructure that never runs proves nothing.

If a conversation goes toward infrastructure, the better move is a short written note on what the production topology would be and why the portfolio version differs. That reads as judgment. A stack diagram full of services that were never deployed reads as the opposite.

Clay integration

Clay is used to build the evaluation corpus, on a 14-day Growth trial. Selecting and filtering the corpus is a real step in this project, and assembling a filtered list of entities with enriched attributes is exactly what Clay is for.

The design rule: **Clay produces an artifact, it is not a runtime dependency.** The corpus is built once, exported, and committed to the repo. Nothing in the demo path calls Clay, so nothing breaks when the trial ends on day 15.

What is used

- **Growth trial (14 days).** HTTP API enrichment columns, Use AI columns, waterfalls, conditional run logic. This is the part that demonstrates product fluency.
- [clay-run/agent-plugins](#). Their public repo (130 stars, 22 forks, a CONTRIBUTING.md), holding Clay's agent skills and the `clay` CLI for Claude Code, Codex, and Cursor. A permanent demo target and a place to contribute.
- [developers.clay.com](#). Searches, routines, audiences, signals. Tables are marked Enterprise-only.

Building the corpus in Clay

1. **Seed the table.** One row per candidate Python repository. Trial row cap is 50 per table, which comfortably fits a 12 to 20 repo corpus.
2. **Cheap filters first.** HTTP API column against the GitHub API for stars, open issue count, primary language, and whether Actions is in use. These are single calls and they eliminate most candidates.
3. **Expensive filter second.** A further HTTP API column counting resolvable `Fixes #N` issue-to-PR links, which is the actual selection criterion. Gated on the cheap filters passing, so it only runs on survivors.
4. **Use AI column for domain.** Classify each surviving repo by domain, so the corpus is not twelve web frameworks. Diversity in the corpus is what stops the result being an artifact of one codebase style.
5. **Export and commit.** The table goes to `corpus/repos.csv` in the repo. The evaluation reads that file. Clay is never called again.

The ordering in steps 2 and 3 is the part worth talking about in an interview. Running the expensive enrichment only on rows that passed the cheap one is credit discipline, and it shows an understanding of the product's cost model rather than just its interface.

What is deliberately avoided

- **No Clay call at demo time.** A webhook that stops firing on day 15 would put the fragile part of the stack directly in the demo path.
- **No Claygent for deterministic lookups.** Repository metadata comes from the GitHub API through an HTTP API column, not from an AI web agent. Spending Claygent on a structured API call is the credit-burn pattern Clay's own support forum fields complaints about.

- **No Clay table as the project’s data store.** The corpus is a selection step with a CSV output, not a database.

Open question to resolve first

Start the trial in week 1, not now. It runs 14 days from activation and the corpus build is the only thing that needs it.

Budget the credits. Clay’s own pages disagree on whether the trial grants 1,000 or 2,000 credits, and on whether the trial row cap is 50 or 200. Run the HTTP API columns before the Use AI column, and check the actual allowance in the dashboard on day one.

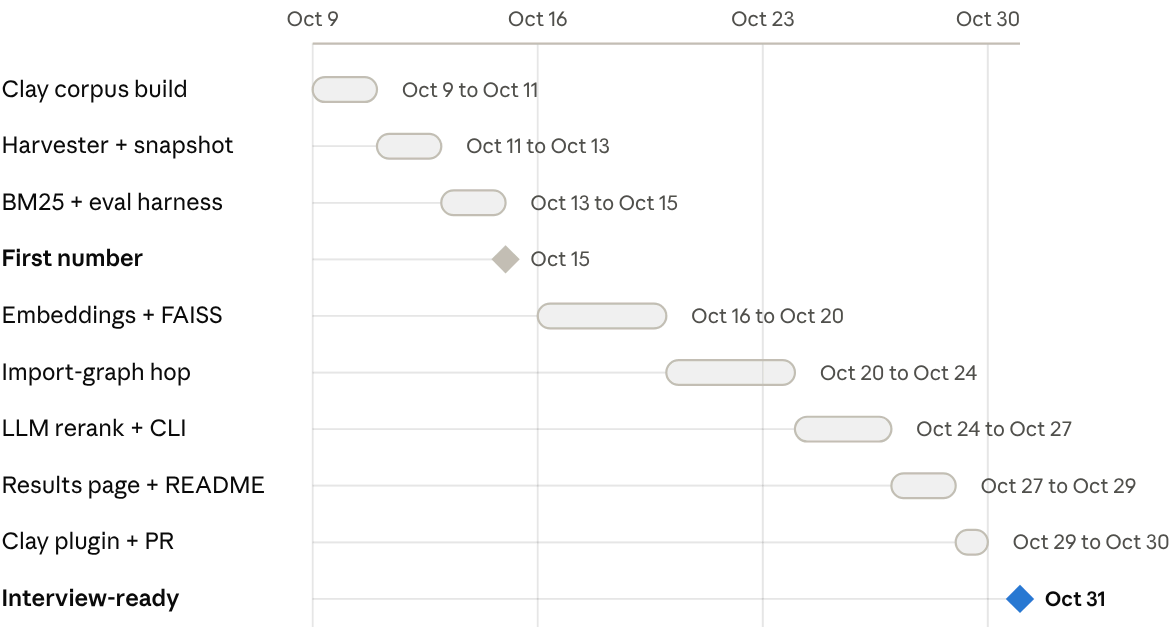
Capture the evidence while it is live. A 90-second screen recording of the table building, the CSV export, and a screenshot of the column configuration. These go in the README and do not expire. Evidence of use is the signal; a live dependency is not.

Test the API on Free first. Clay’s FAQ implies API access is Enterprise-only, while the developer platform page flags only Tables as Enterprise. These conflict. Confirm before planning around the CLI.

Milestones

The first number lands in week one, before any model work

Three weeks part-time, October 2026



Build schedule · 3 weeks, 2 milestones

The ordering is deliberate on two counts. The Clay corpus build comes first, because the Growth trial runs 14 days from activation and nothing later needs it. The BM25 baseline comes before any embedding work, so there is a measured number by day 7. If BM25 cannot be beaten, that is known with two weeks left to reframe the project around the finding.

Week by week

Week	Focus	Done when
1	Clay corpus build on the Growth trial, issue-to-PR harvester, repo snapshotting, BM25 baseline, eval harness	<code>corpus/repos.csv</code> is committed with the Clay recording captured, a recall@10 number exists for BM25 on at least 2 repos, and the leakage test passes
2	Embeddings, FAISS index, import-graph expansion, rank fusion	The ablation table has 4 rows and the margin over BM25 is known
3	LLM rerank, CLI polish, results page, README, Clay plugin and PR	The demo runs on a repo picked by someone else

Cut list, in order

If time runs short, these go first. The project stays defensible after every cut except the last one.

1. TypeScript support. Python only.
2. The LLM rerank stage. Fusion output is already a ranked list.
3. The results page. A markdown table in the README works.
4. The held-out repos. Report on fewer repos and say so.
5. The Clay plugin and PR in week 3. The corpus build in week 1 already carries the product-fluency signal.
6. **Never cut:** the BM25 baseline, the snapshot-leakage test, or the ablation. Those three are what make the number mean anything.

Risks and open questions

Risk	Likelihood	Mitigation
Embeddings do not beat BM25	High	This is the expected outcome at first, and it is survivable. The deliverable becomes the ablation showing lexical retrieval wins on code until import-graph expansion is added, with numbers. Build BM25 in week one so there is time to frame the finding.
Import-graph expansion adds noise instead of signal	Medium	Cap expansion depth at 1 hop and weight graph neighbors below direct hits. If it still hurts, report that: a negative result with a clean ablation is a real contribution.
Scope creep into patch generation	Medium	Explicitly a non-goal. The measurable claim is retrieval quality and nothing else.
Repo snapshotting is subtly wrong and leaks the fix	Medium	Highest-consequence bug in the project, since it silently inflates every number. Write a test that asserts the gold-patch files differ from their post-fix contents in the indexed snapshot.
SWE-bench repos are overrepresented in model training data	Medium	A known concern with this benchmark. The self-harvested set from other repos is the control. Report both separately, never pooled.
Free LLM tier limits change mid-build	Low	The provider-fallback layer handles it. Local embeddings mean the core path needs no network at all.
Three weeks part-time is not enough	High	See the cut list in the milestones section. The week-two state is already demoable.

Open questions

- **Which Clay posting is this aimed at?** A "Software Engineer, Developer Experience (AI)" listing was found whose responsibilities (monorepo tooling, validating agent-generated changes, exposing codebase context to agents) match this project tightly. Confirm which role is being applied to, since it changes how the project is framed.
- **Does Clay sponsor visas for new-grad roles?** Unanswered. Worth asking the recruiter before committing three weeks to a company-specific project.
- **Does the Clay API work on the Free plan?** See the Clay integration section. Test before building against it.
- **Python only, or add TypeScript?** Depends on week-two progress. Clay's stack is TypeScript-heavy, so a TypeScript repo in the evaluation set would land well, but Python first is the right order.

Success criteria

Shipped

- ☐ CLI runs end to end on an arbitrary public Python repo in under 60 seconds
- ☐ Evaluated on SWE-bench Verified and on at least 4 self-harvested repos, 2 of them held out
- ☐ Recall@10 reported against all four baselines, with the ablation table published
- ☐ Snapshot-leakage test passing
- ☐ Static results page live on GitHub Pages
- ☐ README states the method, the number, and the metric's limitation in the first three paragraphs
- ☐ One substantive issue or PR opened on `clay-run/agent-plugins`
- ☐ Clay corpus build recorded, column config screenshotted, `corpus/repos.csv` committed

Interview-ready

The project clears the bar when all three of these are true.

1. **The number survives a hostile question.** "How do you know the fix wasn't already in your index?" has a one-sentence answer and a test to point at.

2. **The failure modes are documented, not hidden.** The ablation includes configurations that lost. The README names the metric's lower-bound problem before a reader finds it.
3. **It runs on input chosen by someone else.** If it only works on a prepared repo, it is a screenshot, not a system.

What this does not need to be

It does not need to beat the state of the art. It does not need to be novel research. It needs to be a correctly measured system with a defensible number, built by one person in three weeks, that runs live. That is rarer in a new-grad portfolio than novelty is.